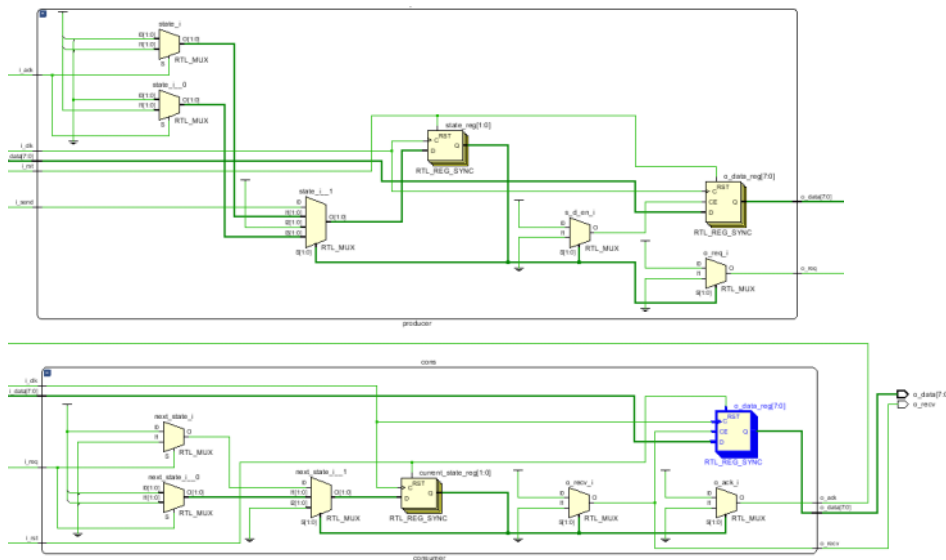
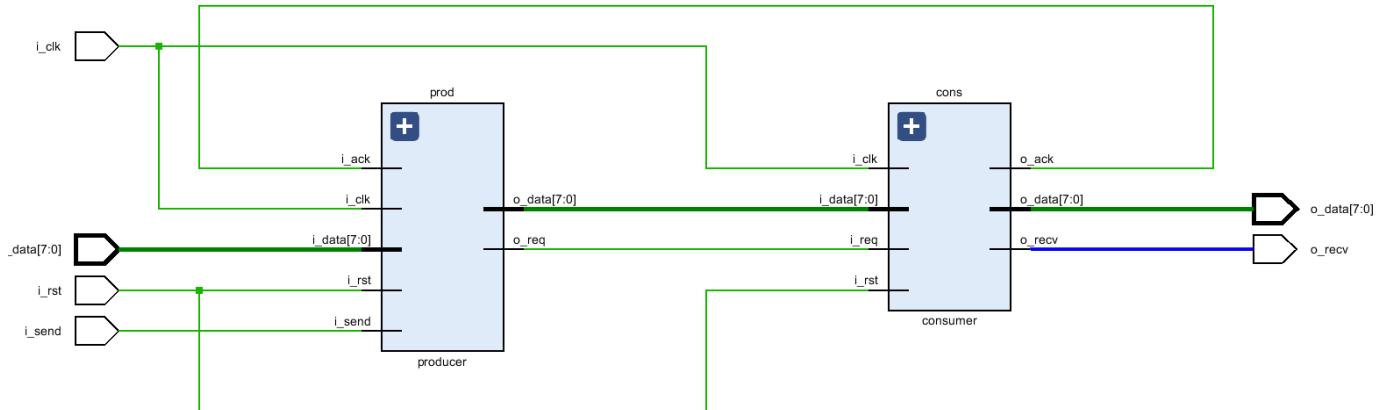


## Tutorial 2

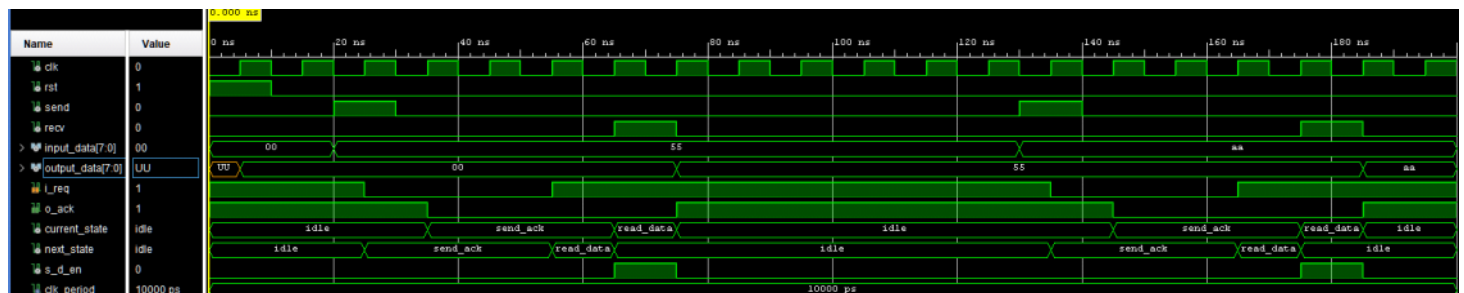
Friday, September 11, 2026 12:10 PM

### Part1:



The first 2 muxes take in either req or ack and change the state. Then those states are fed into another mux that uses a register with the current state as its select signal. That register also takes in the output of the last mux to replace the current state when the next rising edge of the clock cycle hits. The current state is also used for 2 other muxes that handle ack and rcv in consumer, and d\_en and req for producer. Lastly, d\_en and rcv, are used as inputs for the clocked register that stores o\_data.

### PART2:



At the start rst is set high to keep everything at idle. Then when send gets set high the input data is loaded into the bus ready for req and ack to be set low. First req is set low which sets consumer into send\_ack, which sets ack low. That moves producer into send\_data state, then into the hold\_data which sets req high. This causes consumer to switch states to read\_data and set rcv and d\_en. This loads the data into the output bus. Then everything is repeated for the next data byte.

### PART3:

INFO: [Synth 8-802] inferred FSM for state register 'state\_reg' in module 'producer'  
INFO: [Synth 8-802] inferred FSM for state register 'current\_state\_reg' in module 'consumer'

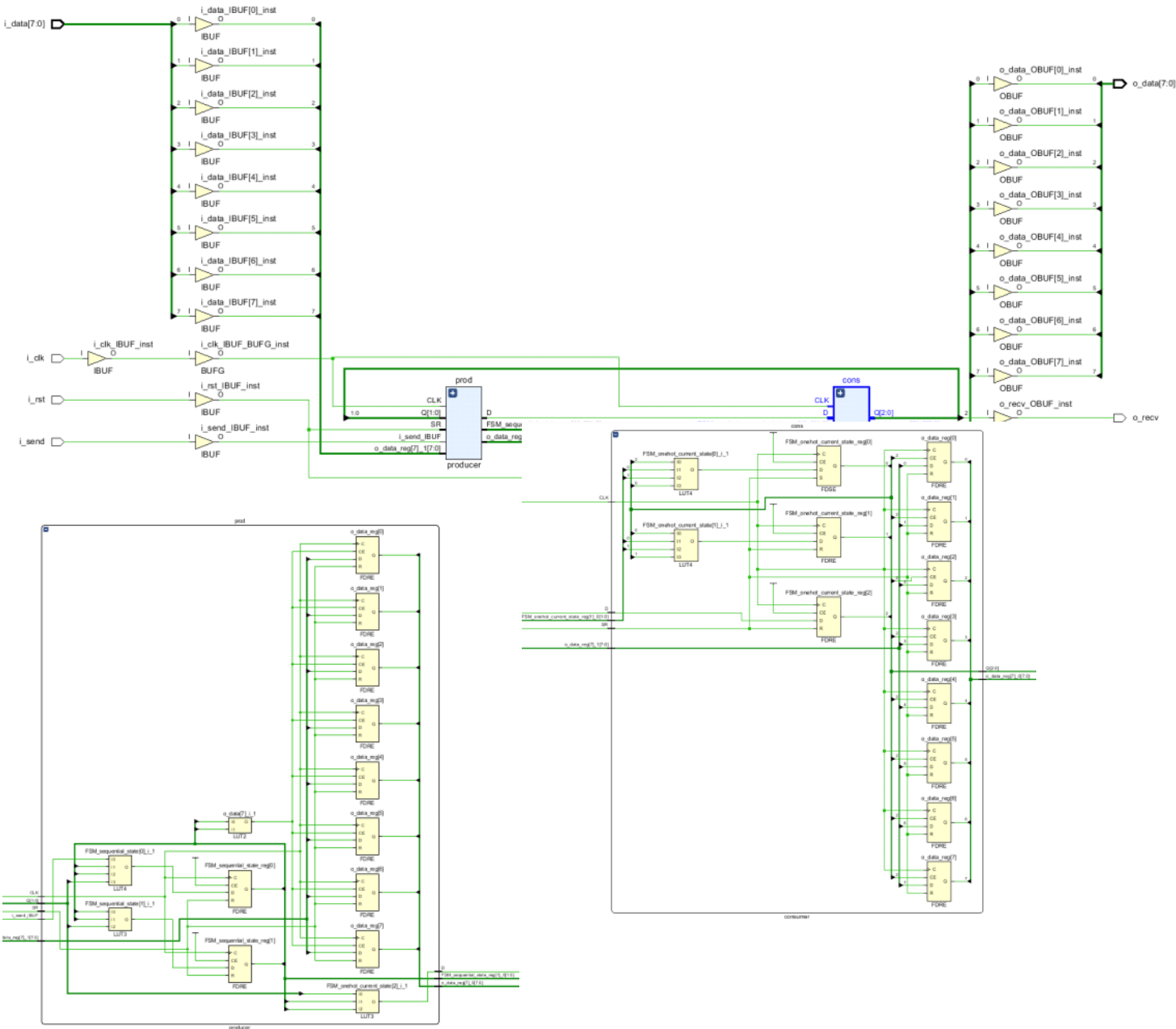
| State     | New Encoding | Previous Encoding |
|-----------|--------------|-------------------|
| idle      | 00           | 00                |
| ack_wait  | 01           | 01                |
| send_data | 10           | 10                |
| hold_data | 11           | 11                |

INFO: [Synth 8-3354] encoded FSM with state register 'state\_reg' using encoding 'sequential' in module 'producer'

| State     | New Encoding | Previous Encoding |
|-----------|--------------|-------------------|
| idle      | 001          | 00                |
| send_ack  | 010          | 01                |
| read_data | 100          | 10                |

INFO: [Synth 8-3354] encoded FSM with state register 'current\_state\_reg' using encoding 'one-hot' in module 'consumer'

As can be seen Producer uses the sequential encoding while Consumer uses one-hot.



## PART4:

### One Hot:

INFO: [Synth 8-802] inferred FSM for state register 'state\_reg' in module 'producer'

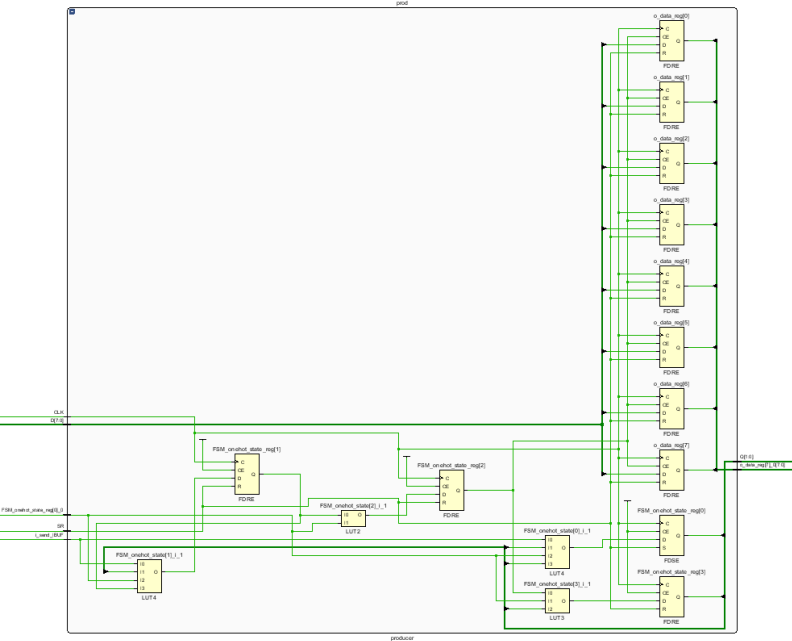
INFO: [Synth 8-802] inferred FSM for state register 'current\_state\_reg' in module 'consumer'

| State     | New Encoding | Previous Encoding |
|-----------|--------------|-------------------|
| idle      | 0001         | 00                |
| ack_wait  | 0010         | 01                |
| send_data | 0100         | 10                |
| hold_data | 1000         | 11                |

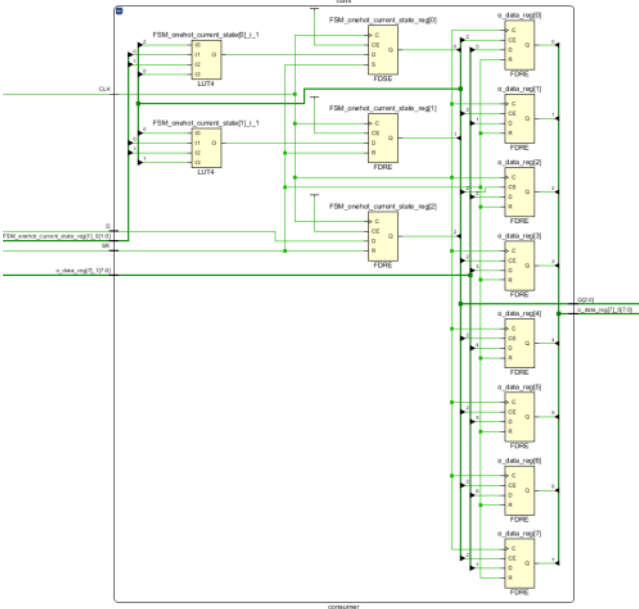
INFO: [Synth 8-3354] encoded FSM with state register 'state\_reg' using encoding 'one-hot' in module 'producer'

| State     | New Encoding | Previous Encoding |
|-----------|--------------|-------------------|
| idle      | 001          | 00                |
| send_ack  | 010          | 01                |
| read_data | 100          | 10                |

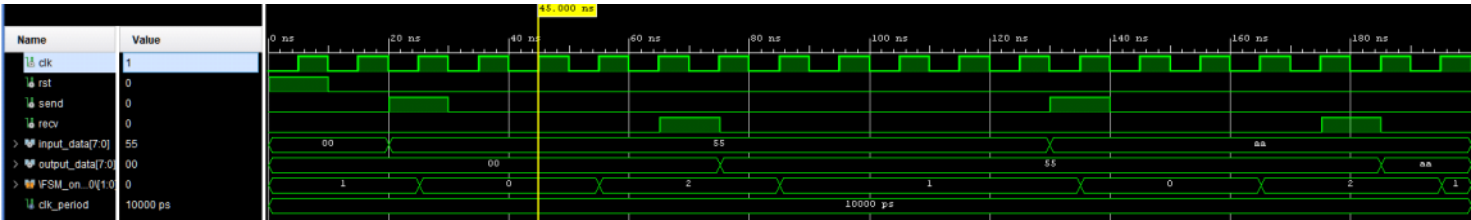
INFO: [Synth 8-3354] encoded FSM with state register 'current\_state\_reg' using encoding 'one-hot' in module 'consumer'



4 Luts, 8 data reg FFs, 6 state reg FFs



2 LUTs, 3 state reg FF, 8 data reg FF



## Sequential:

INFO: [Synth 8-802] inferred FSM for state register 'state\_reg' in module 'producer'

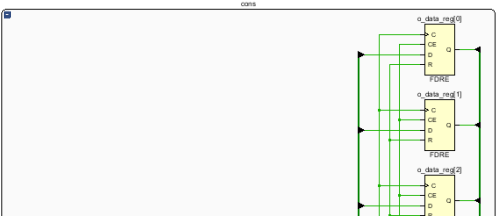
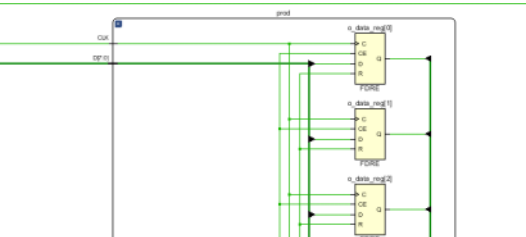
INFO: [Synth 8-802] inferred FSM for state register 'current\_state\_reg' in module 'consumer'

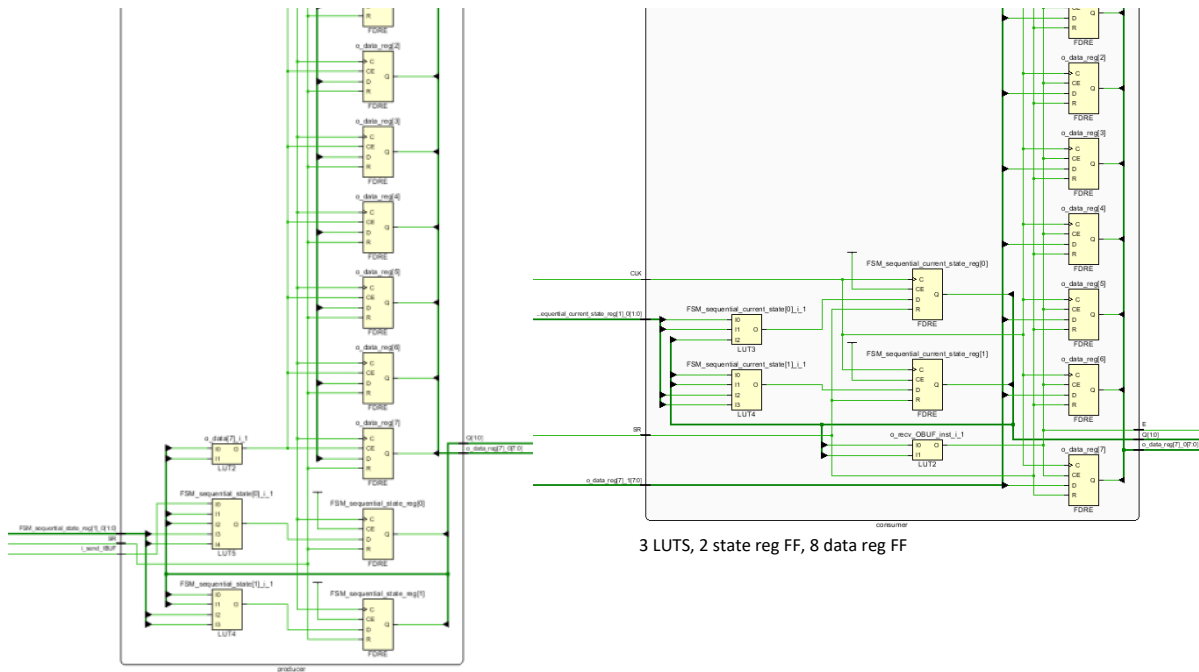
| State     | New Encoding | Previous Encoding |
|-----------|--------------|-------------------|
| idle      | 00           | 00                |
| ack_wait  | 01           | 01                |
| send_data | 10           | 10                |
| hold_data | 11           | 11                |

INFO: [Synth 8-3354] encoded FSM with state register 'state\_reg' using encoding 'sequential' in module 'producer'

| State     | New Encoding | Previous Encoding |
|-----------|--------------|-------------------|
| idle      | 00           | 00                |
| send_ack  | 01           | 01                |
| read_data | 10           | 10                |

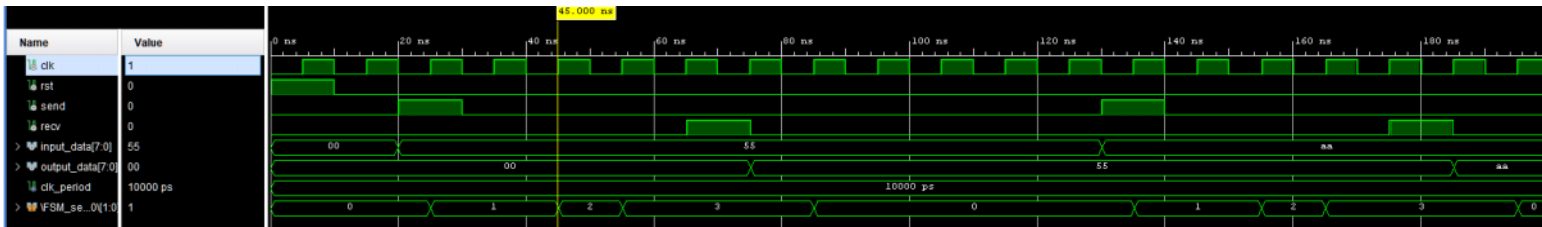
INFO: [Synth 8-3354] encoded FSM with state register 'current\_state\_reg' using encoding 'sequential' in module 'consumer'





3 LUTs, 2 state reg FF, 8 data reg FF

3 LUTs, 2 state reg FF, 8 data reg FF



It seems that sequential encoding uses less LUTs and FFs, this could be because it uses less bytes and therefore the program doesn't need to generate the elements to simulate them.